

Partie 02 — Comprendre, importer et explorer des données

INF-STAR — Programmation statistique avec R

Oskar Laverny

September 20, 2026

Aix-Marseille Université

Objectifs

À la fin de cette partie

Vous saurez :

- reconnaître les types de données courants ;
- créer et indexer des vecteurs ;
- comprendre les valeurs manquantes `NA` ;
- manipuler un `data.frame` ;
- importer un fichier CSV et explorer rapidement son contenu.

Les vecteurs



Un vecteur : une suite de valeurs

```
age <- c(34, 51, 68, 42)
sexe <- c("F", "M", "F", "F")
positif <- c(TRUE, FALSE, TRUE, TRUE)
```

Un vecteur contient des valeurs d'un même type fondamental.

Types utiles au quotidien

```
class(age)  
class(sexe)  
class(positif)
```

Pour ce cours, retenez surtout :

- `numeric / integer` ;
- `character` ;
- `logical` ;
- les dates, que nous verrons comme une classe spécialisée.

Inspecter un objet

```
length(age)  
class(age)  
str(age)
```

`str()` est particulièrement utile car il donne une vue compacte de la structure d'un objet.

Créer des séquences

```
1:10  
seq(from = 0, to = 1, by = 0.2)  
rep(0, times = 5)  
rep(c("A", "B"), times = 3)
```

Ces fonctions évitent d'écrire manuellement de longues suites.

Indexer



Par position

```
age <- c(34, 51, 68, 42)
```

```
age[1]
```

```
age[c(1, 4)]
```

```
age[-2]
```

Les indices commencent à **1** en R.

```
age > 50  
age[age > 50]
```

L'expression `age > 50` produit un vecteur de `TRUE` et `FALSE` utilisé pour sélectionner des valeurs.

Cette idée sera centrale dans tout le cours.

Valeurs manquantes



NA signifie « valeur manquante »

```
x <- c(10, 12, NA, 15)
mean(x)
mean(x, na.rm = TRUE)
```

NA se propage souvent dans les calculs pour éviter de masquer un problème de données.

Tester une valeur manquante

À éviter :

```
x == NA
```

À utiliser :

```
is.na(x)  
sum(is.na(x))
```

Coercition



Mélanger les types

```
c(1, 2, "trois")
```

R doit choisir un type commun : ici, toutes les valeurs deviennent du texte.

C'est une source fréquente de surprise lors de l'import de données mal formatées.

Data frames

Des colonnes décrivant des observations

```
patients <- data.frame(  
  id = 1:3,  
  age = c(34, 51, 68),  
  sexe = c("F", "M", "F")  
)
```

Chaque ligne correspond ici à un patient ; chaque colonne décrit une variable.

Inspecter un data frame

```
head(patients)
names(patients)
dim(patients)
str(patients)
summary(patients)
```

Avec dplyr :

```
dplyr::glimpse(patients)
```

```
patients$age  
patients[["age"]]  
patients[, "age"]
```

Pour débiter, \$ est souvent la syntaxe la plus lisible.

Sélectionner des lignes et colonnes

```
patients[1:2, ]  
patients[, c("id", "age")]  
patients[patients$age >= 50, ]
```

La partie suivante introduira `dplyr`, souvent plus lisible pour ces opérations.

Importer des données



```
patients <- readr::read_csv("patients.csv")
```

Puis immédiatement :

```
dplyr::glimpse(patients)  
summary(patients)
```

Le fichier `source/data/patients.csv` fourni avec ce cours est synthétique.

```
patients <- read.csv("patients.csv")
```

Les deux approches sont valides. `readr::read_csv()` fournit souvent des messages et types plus explicites pour un workflow moderne.

Le package `readxl` permet notamment :

```
readxl::read_excel("donnees.xlsx")
```

Réflexe : vérifier les noms de colonnes et les types juste après l'import.

Facteurs, matrices et listes

Facteurs : variables catégorielles

```
groupe <- factor(c("A", "A", "B", "A"))  
levels(groupe)
```

Les facteurs deviennent utiles lorsque l'ordre ou les niveaux d'une catégorie ont un sens explicite.

Ne transformez pas automatiquement toutes les colonnes texte en facteurs.

```
m <- matrix(1:12, nrow = 3)
m
m[1, 2]
```

Une matrice est utile lorsque toutes les cellules ont le même type, notamment pour certains calculs numériques.

```
x <- list(  
  nom = "Alice",  
  age = 42,  
  mesures = c(12.1, 11.8, 12.4)  
)
```

Une liste peut contenir des objets de natures différentes.

Les data frames sont eux-mêmes construits à partir de listes de colonnes.

Pour aller plus loin



Détails internes : utiles, mais pas prioritaires

Vous rencontrerez parfois :

```
typeof(x)  
attributes(x)  
unclass(x)
```

Ces fonctions aident à comprendre les mécanismes internes de R, mais elles ne sont pas nécessaires pour commencer à analyser des données.

Priorité : savoir **inspecter**, **sélectionner** et **comprendre** ses données.

À retenir



Workflow après un import

```
patients <- readr::read_csv("patients.csv")  
  
dplyr::glimpse(patients)  
summary(patients)  
colSums(is.na(patients))
```

Avant toute analyse, demandez-vous :

- quelles sont les lignes ?
- quelles sont les colonnes ?
- quels sont les types ?
- y a-t-il des valeurs manquantes ou aberrantes ?